

Food Store — TP4

Reportes analíticos asistidos por IA: joins, subconsultas, agregación y ventana

Base de Datos II — UTN · Equipo G · 22/09/2026

5 analíticas medidas con EXPLAIN · Tabla antes/después por nodo de join (Parte 1)

Lectura crítica de un plan multi-join explicado por IA (Parte 2)

Equivalencias de specs de ventana/subconsulta con EXCEPT (Parte 3)

Desafío de la competencia: T5 ×3.5 con MV (Parte 4) · DUIA

Informe de laboratorio — Analíticas con joins, agregación y ventana (TP4, Parte 1)

Materia: Base de Datos II — UTN · **Proyecto:** Food Store · **Fecha:** 22/09/2026 **Base:** `food_store_tp3` (50.006 productos, 20.003 clientes, 200.003 pedidos, 500.014 líneas) **Flujo:** EXPLAIN (ANALYZE, BUFFERS) en caliente → plan real a la IA → propuesta por nodo → se lee y decide → se aplica → se re-mide. Todo lo que no mejora se documenta.

Consultas analíticas medidas (`sql/queries_tp4.sql` , todas cruzan ≥3 tablas)

- **T1** facturación por categoría y mes (detalle+pedido+producto+categoría, ENTREGADO).
- **T2** top 3 productos por categoría según recaudación (detalle+producto+categoría, ventana ROW_NUMBER).
- **T3** top 50 pedidos por importe con cliente y líneas (pedido+cliente+detalle, no CANCELADO).
- **T4** clientes que compraron en todas las categorías (cliente+pedido+detalle+producto+categoría, subconsulta correlacionada).
- **T5** facturación mensual con acumulado de clientes frecuentes (pedido+detalle+cliente, ventana).

1.2 Tabla de resultados (Execution Time real en caliente)

Consulta	Algoritmo de join (antes)	Cambio aplicado	Algoritmo de join (después)	Antes → Después (ms)	Mejora
T1 fact cat/mes	Hash Join (detalle↔pedido) en paralelo + Hash Join ↔producto + Hash Join ↔categoría → Sort externo a disco (3,4 MB) para el GroupAggregate	MV <code>mv_ventas_categoria_mes</code> (materialización analítica): el sort+join sobre 500k líneas no se justifica para un reporte de 39 filas	— (la consulta lee la MV: Seq Scan de 39 filas, sin joins)	540.832 → 0.293	× 1846 ✓
T2 top3 por categoría	Hash Join (detalle↔producto) + Hash Join ↔categoría → Sort externo 8.168 kB a disco (2,4 s) previo al GroupAggregate + WindowAgg	work_mem = 256 MB (sesión) — no hay índice que reduzca un agregado de tabla completa; el cuello era el derrame del sort, verificado en el plan (Sort Method: external merge Disk: 8168kB)	Hash Join (igual) — no cambia el algoritmo; el sort pasa de external merge (disco) a quicksort (memoria)	3096.592 → 725.510	× 4.3 (rango medido 725–965) ✓
T3 top50 pedidos	Nested Loop (pedido↔cliente, con Memoize) sobre los ~167 mil pedidos + Merge Join	Reescritura: agregar detalle por pedido_id primero y adjuntar el cliente solo a los 50 elegidos (<code>sql/queries_tp4_opt.sql</code>)	Merge Join (detalle↔pedido) + Nested Loop solo sobre los 50 pedidos del top (cliente: 50 loops, no 166.857)	791.681 → 485.145	× 1.63 ✓

Consulta	Algoritmo de join (antes)	Cambio aplicado	Algoritmo de join (después)	Antes → Después (ms)	Mejora
	(↔detalle, 500k líneas) + Incremental Sort de 12.645 grupos por cliente	— los montos viven en la línea, el join con clientes no necesita el 100 %			
T4 clientes todas las categorías	Nested Loop Anti de 3 niveles (por cliente → por pedido → por línea; 133.542 lookups a producto con filtro <code>categoria_id</code> , 400.626 buffers)	Índice covering <code>ix_producto_cat_cover ON producto (id) INCLUDE (categoria_id)</code> para dejar de tocar el heap en el lookup por línea	El índice se usa (Index Only Scan <code>ix_producto_cat_cover</code>) pero el plan sigue igual de costoso	553.184 → 546.758	×1.0 — sin mejora , se documenta: el cuello es el bucle correlacionado de 133k ejecuciones, no el heap de producto
T5 mensual acumulado	Hash Join (detalle↔pedido, vía <code>ix_pedido_estado</code>) + Nested Loop cliente + CTE materializada 2 veces a disco y re-leída desde disco (216 ms) + Sort externo	MV <code>mv_fact_cliente_mes</code> (cliente, mes): el running total es métrica de dashboard, re-consultada; además se descartó una reescritura que parecía equivalente pero cambiaba la semántica (ver §1.3)	Hash Join/Scan sobre la MV (sin re-scanear 500k líneas ni materializado doble)	825.829 → 236.047	×3.5 ✓

1.3 Criterio de aceptación (aplicado, y la equivocación que se destapó)

- **Se aceptaron** T1 (MV), T2 (work_mem), T3 (reescritura), T5 (MV): cada una se explica por un nodo concreto del plan y se confirmó con medición.
- **Se probó y rechazó OPT-T5-A**: mover `HAVING COUNT(*) >= 5` dentro de la CTE parecía equivalente, pero cambió el **filtro de borrado lógico/negocio**: pasó de "≥5 meses con ventas" a "≥5 líneas en un mes" → 46.322 filas (original) vs 10.211 (intento). Se detectó comparando conteos; la reescritura corregida sobre la MV preserva la semántica exacta (verificado EXCEPT 0/0).
- **Se aplicó pero no mejoró** T4 (índice covering): se documenta; el plan mostró que el índice sí se usa pero no es el cuello.
- **T2 — cambio de algoritmo de join**: el índice NO cambia el algoritmo (sigue Hash Join); el ajuste de `work_mem` cambia el nodo Sort (external merge → quicksort). Se documenta porque la mejora nació de un knob de entorno, no de un índice, y eso se explica en la defensa oral.

Planes completos (verbatim): `docs/planes4/*_antes.txt`, `docs/planes4/*_despues.txt` (T2 bajo work_mem: `T2_top3_categoria_despues_wmem.txt`).

Ejercicio de lectura crítica: plan con JOIN múltiple interpretado por IA (TP4, Parte 2)

Materia: Base de Datos II — UTN · **Proyecto:** Food Store · **Fecha:** 22/09/2026 **Plan analizado:** docs/planes4/T3_top50_pedidos_antes.txt (top 50 pedidos por importe, con cliente y líneas)

Contexto

El plan de T3 combina tres algoritmos de join reales: un **Nested Loop** (con Memoize), un **Merge Join** y un **Incremental Sort** para el agregado. Se le pasó el texto del plan (sin otro contexto) a la IA para que lo explicara "nodo por nodo", con foco en qué tabla es externa e interna en cada join. La consigna exige contrastar: la IA tiende a leer el costo estimado de un nodo como tiempo y a confundir el orden de las tablas.

3.1 ... explicación generada por la IA (textual)

"El plan primero ordena las 500.014 líneas de detalle_pedido por pedido_id con un Sort, y después un Merge Join mezcla esas líneas con la salida del Nested Loop. En el Nested Loop la tabla **externa es cliente** e **interna es pedido**, y el Memoize guarda en memoria los pedidos ya vistos para no releerlos. El costo estimado del Merge Join (51.376) equivale a unos 51 milisegundos de ese paso; el 30% del tiempo total (791 ms) es el Incremental Sort que ordena a los clientes. Como la consulta pide 50 pedidos, el plan solo procesa los mejores 50."

3.2 Contraste contra el plan real

#	Afirmación de la IA	¿Correcta?	Corrección · evidencia del plan real
1	"Se ordena detalle_pedido con un Sort antes del Merge Join"	X	No hay ningún nodo Sort sobre detalle: el Merge Join lee las líneas desde el índice Index Scan using uq_detalle_pedido_producto (ya ordenado por pedido_id), cost=0.42..24870.51. Lo único que ordena es el Incremental Sort con Sort Key: pe.id, c.nombre, c.apellido (por cliente), no las líneas.
2	"En el Nested Loop la tabla externa es cliente y la interna es pedido"	X	Es al revés: la externa es pedido (Index Scan using pedido_pkey ... Filter estado <> 'CANCELADO', 166.857 filas) y la interna es cliente (cliente_pkey, vía Memoize). El plan lo muestra por orden de anidación: -> Nested Loop / -> pedido / -> Memoize -> cliente.
3	"El Memoize guarda en memoria los pedidos para no releerlos"	X	Memoize cachea el resultado del lookup a cliente por cliente_id: Cache Key: pe.cliente_id, con Hits: 146860, Misses: 19997 (19997 clientes distintos resueltos y reutilizados). No guarda pedidos.
4	"El costo estimado del Merge Join (51.376) equivale a ~51 milisegundos"	X	El cost de EXPLAIN es adimensional (ponderación de I/O + CPU del optimizador), no tiempo. El tiempo real del Merge Join se lee de actual time: 0.239..399.562 ms.

#	Afirmación de la IA	¿Correcta?	Corrección · evidencia del plan real
5	"El 30% del tiempo total es el Incremental Sort que ordena clientes"	Parcial	El Incremental Sort (<code>actual time=0.385..511.018 ms</code>) sí es un nodo caro, pero también lo es el Merge Join (~399 ms) y el GroupAggregate (~350 ms). Ninguno es "el 30%"; la afirmación de reparto es inventada, no viene del plan.
6	"Como pide 50 pedidos, el plan procesa solo los mejores 50"	X	El plan recorre 166.857 pedidos y 500.014 líneas (y agrega por <code>Group Key: pe.id, c.nombre, c.apellido</code>) antes del <code>Limit 50</code> . El <code>Limit (top-N heapsort)</code> es el último nodo.

Conclusión: 5 de 6 afirmaciones resultaron falsas o parciales. Las dos imprecisiones más graves son la **inversión de tabla externa/interna** del Nested Loop (afecta la lectura del plan para proponer optimizaciones) y la **confusión costo estimado vs tiempo real** (afecta la interpretación de cualquier propuesta). Regla para el informe y la defensa: citar siempre el nodo textual (`-> Nested Loop`, `Hits/Misses`, `actual time`) y nunca la interpretación libre de la IA.

3.3 Qué significó esto para las propuestas

Esta lectura crítica evitó aceptar la propuesta "colocar un índice en `cliente(apellido)` porque el Incremental Sort es el 30%": el nodo Interno real del Incremental Sort era la **clave de agrupación** (`pe.id, c.nombre, c.apellido`) **de salida del Merge Join**, y el verdadero cuello era el volumen que atraviesa el join (166k × 500k), no la resolución de clientes. Por eso la reescritura de T3 (agregar por `pedido_id` primero y adjuntar el cliente solo a los 50 elegidos) atacó el nodo correcto.

FOOD STORE — TP4 Parte 3: Equivalencia de consultas con EXCEPT

DB de laboratorio: **food_store_tp3** (50.006 productos, 20.003 clientes, 200.003 pedidos, 500.014 líneas). Todas las corridas sobre la copia de trabajo, servidor caliente, PostgreSQL 18.

Nota metodológica: `EXCEPT` es una operación de CONJUNTO sobre la tupla completa (en este caso tuplas con texto/numérico, comparables sin ambigüedad). `A - B` da 0 filas implica que todo resultado de A está en B; `B - A` da 0 implica que todo resultado de B está en A; juntos prueban que los conjuntos de resultados son idénticos.

ESPEC (a) — Ranking con función de ventana

Enunciado preciso. "Para cada cliente con al menos un pedido ENTREGADO, mostrar apellido, nombre, email, el total gastado (suma de `cantidad * precio_unitario` de las líneas de sus pedidos ENTREGADO) y su puesto en un ranking de mayor a menor gasto. Los que empatan en gasto comparten puesto y los siguientes quedan saltados (RANK clásico). Orden final por puesto y por id ascendente. No usar `SELECT *`."

Dato de entrada. 19.882 clientes cumplen la condición (tienen ≥ 1 pedido ENTREGADO).

Versión (A) — generada por IA, con `RANK() OVER`

```
SELECT apellido, nombre, email, total,
       RANK() OVER (ORDER BY total DESC) AS puesto
FROM totales;
```

Intento (B₁) — "RANK sin ventana" con `COUNT(DISTINCT)` **(NO equivalía)**

```
SELECT apellido, nombre, email, total,
       1 + (SELECT COUNT(DISTINCT t2.total)
            FROM totales t2 WHERE t2.total > t.total) AS puesto
FROM totales t;
```

Error destapado por EXCEPT. `A - B1 = 19.743` filas (y por simetría `B1 - A = 19.743`). La causa es semántica: `RANK` cuenta **filas** con valor mayor (los empates comparten puesto porque ninguno es "mayor"), mientras `COUNT(DISTINCT total)` cuenta **valores** distintos, que es exactamente la definición de `DENSE_RANK`. Con 19.882 clientes y pocos miles de totales distintos (muchas sumas repetidas), casi todas las filas quedaban con un puesto menor en B₁. El mecanismo de verificación (no la relectura) dejó en evidencia que la "alternativa sin ventana" no era equivalente.

Versión (B) corregida — con `COUNT(*)` **de filas mayores (equivalente)**

```
SELECT apellido, nombre, email, total,
       1 + (SELECT COUNT(*)
            FROM totales t2 WHERE t2.total > t.total) AS puesto
FROM totales t;
```

Verificación con EXCEPT (corrida definitiva)

Chequeo	Filas
filas_A	19.882
filas_B	19.882
especA_A - B	0
especA_B - A	0

La versión corregida usa la definición clásica de RANK por conteo de filas estrictamente mayores; la verificación confirma igualdad de conjuntos en ambas direcciones.

ESPEC (b) — Subconsulta correlacionada

Enunciado preciso. "Para cada producto vigente, mostrar id, nombre, precio y un indicador SÍ/NO de si su precio es MAYOR que el promedio de los precios de los productos vigentes de su propia categoría. No usar `SELECT *`."

Dato de entrada. 50.006 productos, todos vigentes, repartidos en 3 categorías.

Versión (A) — generada por IA, subconsulta escalar correlacionada

```
SELECT p.id, p.nombre, p.precio,
       CASE WHEN p.precio > (SELECT AVG(p2.precio)
                             FROM producto p2
                             WHERE p2.categoria_id = p.categoria_id
                             AND p2.activo)
       THEN 'SI' ELSE 'NO' END AS sobre_promedio
FROM producto p
WHERE p.activo;
```

Versión (B) — alternativa: `AVG OVER` con partición por categoría

```
SELECT id, nombre, precio,
       CASE WHEN precio > AVG(precio) OVER (PARTITION BY categoria_id)
       THEN 'SI' ELSE 'NO' END AS sobre_promedio
FROM producto p
WHERE activo;
```

Ambas computan el mismo promedio por categoría (identidad `AVG(...) OVER (PARTITION BY categoria_id)` $\equiv$ promedio del grupo de la subconsulta correlacionada, dado que la partición se define exactamente por `categoria_id` y el filtro `activo` es el mismo en ambos lados).

Verificación con EXCEPT (corrida definitiva)

Chequeo	Valor
<code>filas_A</code> / <code>filas_B</code>	50.006 / 50.006
<code>si_A</code> / <code>si_B</code>	25.046 / 25.046
<code>especB_A</code> - <code>B</code>	0
<code>especB_B</code> - <code>A</code>	0

Conclusión

- La especificación (a) entregó **un caso real de inconsistencia semántica detectado por EXCEPT**: la versión "artesanal" con `COUNT(DISTINCT total)` implementaba `DENSE_RANK`, no `RANK`. El conteo directo de filas con `COUNT(*)` sí es equivalente y quedó verificado 0/0 en ambas direcciones.
- La especificación (b) fue equivalente desde el primer intento (0/0 en ambas direcciones), confirmando que `AVG OVER (PARTITION BY ...)` y la subconsulta escalar correlacionada por el mismo grupo producen resultados idénticos también en un conjunto de 50.006 filas.
- En todos los casos la verificación se hizo contra el conjunto completo de resultados (no muestras), por lo que el criterio vale sin suposiciones estadísticas.

Script de verificación: `sql/equivalencias_tp4.sql` (reproducible sobre `food_store_tp3`).

Registro del desafío de la competencia (TP4, Parte 4)

Equipo G · Proyecto: Food Store · **Fecha:** 22/09/2026 **Herramienta de IA:** OpenCode (CLI, modelo *big-pickle*) *Foco curricular:* * joins + agregación + funciones de ventana en una consulta analítica real sobre la copia `food_store_tp3`.

Consulta fija de la competencia

Reporte con **3 tablas** (pedido, detalle_pedido, cliente), **2 joins**, **agregación** (`SUM`, `COUNT`, filtro `HAVING`) y **función de ventana** (running total) — la métrica de "clientes frecuentes", definida como clientes con **5 o más meses** con ventas ENTREGADO:

```
WITH compras AS (  
    SELECT pe.cliente_id, date_trunc('month', pe.fecha) AS mes,  
           SUM(d.cantidad * d.precio_unitario) AS facturado  
    FROM pedido pe  
    JOIN detalle_pedido d ON d.pedido_id = pe.id  
    WHERE pe.estado = 'ENTREGADO'  
    GROUP BY pe.cliente_id, date_trunc('month', pe.fecha)  
)  
SELECT c.id, c.nombre || ' ' || c.apellido AS nombre,  
       to_char(m.mes, 'YYYY-MM') AS mes,  
       m.facturado,  
       SUM(m.facturado) OVER (PARTITION BY c.id ORDER BY m.mes) AS acumulado  
FROM compras m  
JOIN cliente c ON c.id = m.cliente_id  
WHERE c.id IN (  
    SELECT cliente_id FROM compras  
    GROUP BY cliente_id HAVING COUNT(*) >= 5  
)  
ORDER BY c.id, m.mes;
```

Resultado: 46.322 filas (8.017 clientes frecuentes).

Diagnóstico del plan real (EXPLAIN ANALYZE, en caliente)

El plan "antes" no está limitado por un join: los dos `Hash Join` terminan en ~220 ms. El costo dominante es que la **CTE `compras` se materializa a disco y se lee DOS veces** (`Storage: Disk Maximum Storage: 4096kB, temp read=1634 written=2372`), porque el `HashAggregate` de 82.532 (cliente, mes) derrama en 5 batches (`Batches: 5 Disk Usage: 7880kB`), y además hay un `Sort` de 4,6 MB (`Sort Method: external merge Disk: 2616kB`) antes del `WindowAgg`. Todo eso se paga por cada corrida del reporte.

Estrategia aplicada (materialización analítica)

1. **Propuesta de IA aceptada (OPT-T5):** materializar el agregado `(cliente, mes, facturado)` en una vista materializada, porque es un dashboard que se consulta repetidamente y solo cubre pedidos ENTREGADO (registros cerrados). La consulta queda **semánticamente idéntica**: la MV es exactamente la CTE del original; el filtro de frecuentes (`>= 5 meses`) y el running total se re-aplican sobre la MV.
2. **Índice de soporte** `ix_mv_fact_cliente_mes (cliente_id, mes)` para la partición del `WindowAgg` y el join con cliente, más `ANALYZE`.

```
CREATE MATERIALIZED VIEW mv_fact_cliente_mes AS
SELECT pe.cliente_id, date_trunc('month', pe.fecha) AS mes,
       SUM(d.cantidad * d.precio_unitario)          AS facturado
FROM pedido pe
JOIN detalle_pedido d ON d.pedido_id = pe.id
WHERE pe.estado = 'ENTREGADO'
GROUP BY pe.cliente_id, date_trunc('month', pe.fecha)
WITH DATA;

CREATE INDEX ix_mv_fact_cliente_mes ON mv_fact_cliente_mes (cliente_id, mes);
ANALYZE mv_fact_cliente_mes;
```

Medición final (EXPLAIN (ANALYZE, BUFFERS), en caliente)

Fase	Nodos dominantes	Execution Time real
Antes	CTE materializada a disco + leída 2× (HashAggregate con Batches: 5, Disk 7880kB) + Sort external merge 2616kB + WindowAgg	825.829 ms
Después	2× Seq Scan sobre MV (82.532 filas) + Hash Joins en memoria + Sort de 2616kB + WindowAgg	236.047 ms

Mejora: $825.829 / 236.047 = \times 3.5$, con buffers compartidos $29.971 \rightarrow 1.422$ y sin escritura de batches a disco en el agregado.

Detalle del plan ganador

```
WindowAgg  (cost=6416.01..7123.01 rows=25709)
  Window: w1 AS (PARTITION BY c.id ORDER BY m.mes)
    -> Sort  Sort Key: c.id, m.mes
    -> Hash Join  (m.cliente_id = c.id)          - Seq Scan cliente (20.003)
        -> Hash Join  (m.cliente_id = mv.cliente_id)
            -> Seq Scan on mv_fact_cliente_mes (82.532 filas)
            -> HashAggregate  (Group Key: cliente_id, Filter: count(*) >= 5)
Execution Time: 236.047 ms
```

La MV elimina el doble materializado a disco y el re-scan de 500.014 líneas; el plan completo quedó en `docs/planes4/T5_fact_mensual_acum_antes.txt` y `docs/planes4/T5_fact_mensual_acum_despues.txt`.

Transparencia: qué se probó y se descartó

- **OPT-T5-A (intento A, RECHAZADO):** mover `HAVING COUNT(*) >= 5` dentro de la CTE (agrupando por mes) *parecía* equivalente pero **cambió la semántica**: "frecuente" pasó de " ≥ 5 meses" a " ≥ 5

líneas en un mes" (46.322 → 10.211 filas). Fue detectado comparando conteos y se descartó.

- **Ajuste de `work_mem`:** probado para el Sort en T2 (antes 3.096 ms → 965 ms con 256 MB de sesión); para T5 no era la palanca correcta: el cuello es la materialización doble de la CTE y 8.2 MB de batches, y `work_mem` de sesión no perdura en producción, mientras que la MV sí.
- **Verificación formal:** la equivalencia original ↔ MV se confirmó con `EXCEPT` 0/0 en ambas direcciones (Parte 3, especificaciones de T5 en `sql/equivalencias_tp4.sql` y `docs/informe_equivalencias_tp4.md`).

Scripts: `sql/optimizaciones_tp4.sql` (DDL) y `sql/queries_tp4_opt.sql` (consulta "después", SQL idéntico en semántica al original).

DUIA — TP4: Reportes analíticos asistidos por IA (Food Store)

Materia: Base de Datos II — UTN · **Proyecto:** Food Store · **Fecha:** 22/09/2026

Herramientas usadas

Herramienta	Uso
OpenCode (CLI de IA, modelo *big-pickle)	Generación de las consultas analíticas T1–T5, interpretación de los planes reales, propuesta de índices/vistas materializadas y generación de la segunda versión de cada consulta para verificar equivalencia.
PostgreSQL 18 (local)	Motor donde se midieron los planes y se verificaron las equivalencias (<code>food_store_tp3</code> , 50k productos / 200k pedidos / 500k líneas, de TP3).
psql (CLI)	<code>EXPLAIN (ANALYZE, BUFFERS)</code> , <code>ANALYZE</code> , <code>EXCEPT</code> , MVs e índices de verificación.
Python runner (<code>run_explain_tp4.py</code> / <code>_opt.py</code>)	Correr EXPLAIN sobre <code>sql/queries_tp4.sql</code> / <code>_opt.sql</code> y archivar los planes en <code>docs/planes4/</code> .
Git + gh	Versionado y publicación en el repositorio <code>rayssa-ale/food-store</code> .

Para qué se usó la IA y con qué prompts/specs

Uso	Prompt / especificación aplicados
Consultas analíticas (Parte 1)	«Traé 5 reportes analíticos reales que crucen 3+ tablas con joins, agregación, funciones de ventana y subconsultas correlacionadas» → <code>sql/queries_tp4.sql</code> (T1 facturación por categoría/mes; T2 top-3 por categoría; T3 top-50 pedidos; T4 clientes que compran en todas las categorías; T5 facturación mensual de frecuentes con running total).
Optimización (Parte 1)	Se pasaron los planes reales con el pedido «justificá cada optimización por el nodo exacto que ataca; si el índice no es la palanca correcta, decilo». Salida: OPT-T1 (MV por categoría/mes), OPT-T2 (ajuste de <code>work_mem</code> , no índice), OPT-T3 (reescritura agregar-primero), OPT-T4 (índice covering, hipótesis a medir), OPT-T5 (MV de cliente/mes) y OPT-T5-A (rechazada).
Lectura crítica (Parte 2)	«Explicá en lenguaje natural este plan multi-join» → se contrastó afirmación por afirmación con los nodos textuales; 5 de 6 afirmaciones resultaron falsas o parciales.
Equivalencia (Parte 3)	«Dadas estas 2 specs precisas (ranking con RANK por gasto; productos sobre el promedio de su categoría), generá el SQL y una versión alternativa con la MISMA semántica» → verificado con EXCEPT.
Competencia (Parte 4)	«Optimizá esta consulta fija (joins + agregación + ventana) sobre la copia» → MV + índice de soporte, verificado ×3.5.

Verificación (regla del TP: la IA propone, el motor decide)

1. **Parte 1 (análisis):** T1 540.832→**0.293 ms (×1846, MV)** · T2 3096.592→**725.510 ms (×4.3, work_mem 256 MB de sesión; 64 MB no ayudó)** · T3 791.681→**485.145 ms (×1.63, reescritura)** ·

T4 553.184→546.758 ms **sin mejora aunque el índice covering SÍ se usó (Index Only Scan)** — documentado · T5 825.829→**236.047 ms (×3.5, MV) ✓** Tabla completa en

`docs/informe_analiticas_tp4.md`.

2. **Parte 2 (lectura crítica):** cada afirmación contrastada con nodos textuales del plan (inversión de lado externo/interno del Nested Loop, qué cachea el Memoize, cost ≠ tiempo real, etc.) ✓
3. **Parte 3 (equivalencia):** ESPEC(a): la "alternativa sin ventana" con `COUNT(DISTINCT total)` resultó **NO equivalente — EL EXCEPT LA DESTAPÓ:** `A - B = 19.743` filas (implementaba `DENSE_RANK`, no `RANK`); corregida a `COUNT(*)` de filas mayores → **0/0** env ambas direcciones. ESPEC(b): subconsulta correlacionada vs `AVG OVER(PARTITION BY categoria_id)` → `filas_A=filas_B=50.006`, `si_A=si_B=25.046`, **0/0 ✓** (`sql/equivalencias_tp4.sql`).
4. **Parte 4 (competencia):** T5 fija y su MV, ×3.5, plan real en `docs/planes4/`, registro en `docs/registro_competencia_tp4.md` ✓

Revisión humana aplicada

- Cada propuesta se justificó por el **nodo exacto** del plan real antes de aplicarse (Sort externo a disco, batches de HashAggregate, doble materialización de CTE, lookups repetidos); ninguna se aplicó porque lo dijera la IA.
- Las mediciones vienen de corridas **en caliente** de `EXPLAIN (ANALYZE, BUFFERS)`, transcritas textuales a `docs/planes4/*.txt`.
- La reescritura de T3 y las MVs de T1/T5 se verificaron con **EXCEPT en ambas direcciones** contra el conjunto completo de resultados (0 filas), no por inspección.
- El caso OPT-T5-A (rechazado por cambio de semántica, 46.322 vs 10.211 filas) se documentó como ejemplo de decisión adversa a la propuesta de la IA.
- Trabajo sobre la **copia** `food_store_tp3` (`protocolo_seguridad.md`; backup previo `respaldos/food_store_tp3_20260922_antes_tp4.sql`); las optimizaciones quedan en `sql/optimizaciones_tp4.sql`, no en `schema.sql`.